Jacobians, Hessians, JVPs and VJPs: From First Principles to Backpropagation

> **Who this is for:** someone who knows high-school differentiation and integration, and wants to understand the machinery behind how neural networks learn, without already knowing words like *perturbation*, *tangent*, or *cotangent*.
>
> **How to read it:** the sections build on each other. Each new idea is introduced only after the ones it needs. If a section feels fast, re-read the worked example right below it.

Table of Contents

1. The big picture (and a roadmap)
2. Rebuilding the derivative: "what happens if I nudge the input?"
3. Functions with many inputs and many outputs
4. Partial derivatives and the gradient
5. The Jacobian: the derivative of a vector function
6. The chain rule becomes matrix multiplication
7. The problem: Jacobians are too big
8. JVP: pushing a nudge forward
9. VJP: pulling a sensitivity backward
10. JVP vs. VJP at a glance
11. Tangents vs. cotangents, demystified
12. Forward mode vs. reverse mode: which one, and why
13. Backpropagation is just a chain of VJPs
14. A complete worked backprop example, by hand
15. The Hessian: the derivative of the derivative
16. Hessian-vector products: using curvature without building it
17. Where all this shows up in AI
18. Seeing it in code (JAX and PyTorch)
19. Cheat sheet

---

1. The big picture (and a roadmap)

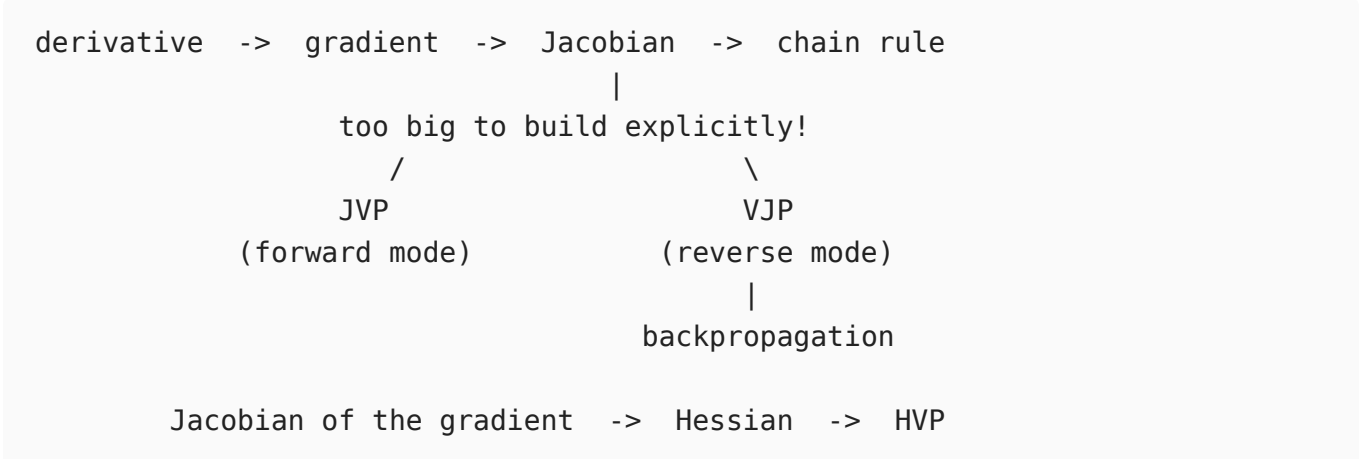
Training a neural network means adjusting millions or billions of numbers (the **parameters** or **weights**) so that a single number, the **loss**, becomes small. The loss measures how wrong the network currently is.

To adjust the weights intelligently, we need to know: **"If I nudge this particular weight a tiny bit, how does the loss change?"** That question is answered by a *derivative*. A network has an enormous number of weights, so we need an enormous number of derivatives, computed quickly. That is the whole story of this document.

Here is the cast of characters, in one sentence each:

Concept	One-sentence meaning
Derivative	How much the output changes per tiny nudge of the input (one input, one output).
Gradient	All the derivatives of a single-number output with respect to many inputs, bundled together.
Jacobian	The same idea when there are many outputs too: a matrix of *all* the derivatives.
JVP	"Jacobian times a vector": what a given nudge of the inputs does to the outputs.
VJP	"Vector times Jacobian": how sensitive a weighted combination of the outputs is to each input.
Backpropagation	A fast way of chaining VJPs from the loss back to the weights.
Hessian	The Jacobian of the gradient: how the slope itself changes, i.e. *curvature*.
HVP	"Hessian times a vector": curvature along a direction, computed without building the Hessian.

The path through the document:



2. Rebuilding the derivative: "what happens if I nudge the input?"

You know that the derivative of $f(x)$ at a point x is the slope of the graph there. Let's re-express this in a way that will generalize cleanly.

2.1 The word "perturbation"

A **perturbation** is just a small nudge or disturbance. If you're at input x and you move to $x + \delta$, where δ (delta) is a tiny number like 0.001, you have *perturbed* the input by δ.

(In some texts, this tiny nudge is called a **tangent** vector, because it points along a direction you could move. Keep that in mind; we'll come back to it in Section 11.)

2.2 The derivative as a "local linear approximation"

Take $f(x) = x^2$ at $x = 3$. Then $f(3) = 9$.

Nudge the input by $\delta = 0.01$:

$$f(3.01) = 9.0601$$

The output changed by 0.0601. Compare to "derivative times nudge": $f'(3) \cdot \delta = 6 \times 0.01 = 0.06$. Very close!

This is the fundamental statement of what a derivative *is*:

$$\boxed{f(x + \delta) \approx f(x) + f'(x)\,\delta \quad \text{for small } \delta}$$

In words: **near a point, the function behaves like a straight line**, and the derivative is the slope of that line. The smaller the nudge, the better the approximation.

The "$\approx$" error shrinks *faster* than δ does (it shrinks like δ^2). That's why we trust this for tiny nudges.

2.3 Why this viewpoint matters

Most people first learn "derivative = slope" or "derivative = limit of a ratio." Those are fine, but for machine learning the most useful reading is:

> **The derivative is a machine that converts an input nudge into an output nudge.**
> Input nudge δ goes in; output nudge $f'(x)\,\delta$ comes out.

Hold on to this. Everything below is this same sentence, upgraded to many dimensions.

3. Functions with many inputs and many outputs

Real functions in AI take lists of numbers (vectors) in and give lists of numbers out. We write:

$$f : \mathbb{R}^n \to \mathbb{R}^m$$

which reads "f takes a vector of n real numbers and returns a vector of m real numbers."

Examples:

Situation	n **(inputs)**	m **(outputs)**
High-school $f(x) = x^2$	1	1
Loss as a function of weights, $L(w_1, \ldots, w_n)$	n (maybe billions)	**1**
A neural network layer $y = Wx + b$	input size	output size
Position of a robot arm given joint angles	number of joints	3

The loss case ($m = 1$) is special and important. **The loss is always a single number**, so that we can say "lower is better."

---

4. Partial derivatives and the gradient

4.1 Partial derivatives

Take a function of two variables: $f(x, y) = x^2 y + y^2$.

A **partial derivative** asks: *if I nudge only one input and freeze the others, how does the output change?*

- $\dfrac{\partial f}{\partial x}$: treat y as a constant, differentiate in x. Result: $2xy$.
- $\dfrac{\partial f}{\partial y}$: treat x as a constant, differentiate in y. Result: $x^2 + 2y$.

(The curly ∂ simply signals "partial": other variables are held fixed.)

4.2 The gradient

Stack the partial derivatives into a vector. That's the **gradient**:

$$\nabla f(x, y) = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix} = \begin{bmatrix} 2xy \\ x^2 + 2y \end{bmatrix}$$

At the point $(1, 2)$: $\nabla f = (4,\ 5)$.

4.3 The gradient and nudges (the multi-dimensional version of Section 2)

Nudge both inputs at once by $\delta = (\delta_x, \delta_y)$. Then:

$$f(\mathbf{x} + \boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \boldsymbol{\delta} = f(\mathbf{x}) + \frac{\partial f}{\partial x}\delta_x + \frac{\partial f}{\partial y}\delta_y$$

Check at $(1, 2)$ with $\delta = (0.01, -0.02)$:

- Exact: $f(1.01, 1.98) = (1.0201)(1.98) + (1.98)^2 = 2.019798 + 3.9204 = 5.940198$. And $f(1, 2)$ $= 2 + 4 = 6$. Change ≈ -0.0598.
- Approximation: $4(0.01) + 5(-0.02) = 0.04 - 0.10 = -0.06$.

Why the gradient matters for learning: the nudge that *increases* f the fastest points in the direction of ∇f. To *decrease* a loss, step in the direction of $-\nabla L$. That's **gradient descent**:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$$

where η is the **learning rate** (step size).

> **Takeaway:** For a function with many inputs and *one* output, the derivative is the gradient. Dotting it with a nudge gives the output change.

---

5. The Jacobian: the derivative of a vector function

Now let there be many outputs. Take

$$f(x_1, x_2) = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix} = \begin{bmatrix} x_1 x_2 \\ x_1 + x_2^2 \\ \sin x_1 \end{bmatrix}$$

Here $n = 2$ inputs and $m = 3$ outputs. Each output f_i has its own gradient (a row of partial derivatives). Stack those rows and you get the **Jacobian matrix** J:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} \end{bmatrix} = \begin{bmatrix} x_2 & x_1 \\ 1 & 2x_2 \\ \cos x_1 & 0 \end{bmatrix}$$

5.1 Shape and indexing: memorize this

For $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is an $m \times n$ matrix:

$$J_{ij} = \frac{\partial f_i}{\partial x_j} \qquad (\text{row } i = \text{which output},\ \text{column } j = \text{which input})$$

- **Rows** correspond to **outputs**.
- **Columns** correspond to **inputs**.
- Row i is the gradient of output f_i (written as a row).
- Column j tells you how *all* outputs respond when you nudge only input x_j.

5.2 The Jacobian is the derivative (same formula as before!)

$$\boxed{f(\mathbf{x} + \boldsymbol{\delta}) \approx f(\mathbf{x}) + J(\mathbf{x})\,\boldsymbol{\delta}}$$

Compare with Section 2: $f(x + \delta) \approx f(x) + f'(x)\delta$. Same sentence! The scalar $f'(x)$ became the matrix J. The Jacobian is **the machine that converts an input-nudge vector into an output-nudge vector**.

5.3 Numeric check

At $\mathbf{x} = (1, 2)$:

$$J = \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ \cos 1 & 0 \end{bmatrix} \approx \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ 0.5403 & 0 \end{bmatrix}$$

Nudge by $\boldsymbol{\delta} = (0.01, 0.00)$ (only x_1 moves):

$$J\boldsymbol{\delta} = (0.02,\ 0.01,\ 0.005403)$$

True values: $f(1, 2) = (2,\ 5,\ 0.841471)$ and $f(1.01, 2) = (2.02,\ 5.01,\ 0.846832)$, so the actual changes are $(0.02,\ 0.01,\ 0.005361)$. The first two match exactly (those outputs are linear in x_1); the third (0.005361 vs. the predicted 0.005403) differs slightly because $\sin$ is curved, which is the leftover error that vanishes as $\delta \to 0$.

5.4 Special cases (to anchor your intuition)

- $m = 1$ (scalar output): J is a single row, the gradient written horizontally ($1 \times n$).
- $n = m = 1$: J is a 1×1 matrix, your old friend $f'(x)$.
- Linear layer $f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$: $J = W$. Always, everywhere. (A linear map's best linear approximation is itself.)
- Elementwise function like ReLU or $\tanh$ applied to each entry: J is **diagonal**, since output i only depends on input i.

---

6. The chain rule becomes matrix multiplication

Neural networks are compositions of functions: $\mathbf{x} \to f \to g \to h \to L$. The chain rule says how to differentiate a composition.

High school: $\frac{d}{dx} g(f(x)) = g'(f(x)) \cdot f'(x)$.

Now: if $\mathbf{y} = f(\mathbf{x})$ and $\mathbf{z} = g(\mathbf{y})$, then

$$\boxed{J_{g \circ f}(\mathbf{x}) = J_g(\mathbf{y}) \cdot J_f(\mathbf{x})}$$

Matrix product, with the *later* function's Jacobian on the **left**.

Why it makes sense (in nudge language): a nudge $\boldsymbol{\delta}$ to $\mathbf{x}$ becomes a nudge $J_f\boldsymbol{\delta}$ to $\mathbf{y}$, which then becomes a nudge $J_g(J_f\boldsymbol{\delta})$ to $\mathbf{z}$. Matrix multiplication is associative, so $J_g(J_f\boldsymbol{\delta}) = (J_g J_f)\boldsymbol{\delta}$.

Shape check: $f : \mathbb{R}^n \to \mathbb{R}^p$ has J_f of shape $p \times n$. $g : \mathbb{R}^p \to \mathbb{R}^m$ has J_g of shape $m \times p$. Product: $(m \times p)(p \times n) = m \times n$.

For a deep network with layers $f_1, f_2, \ldots, f_k$:

$$J = J_k \cdot J_{k-1} \cdots J_2 \cdot J_1$$

A product of many matrices. The *order in which we multiply them* is where all the interesting computer science hides.

---

7. The problem: Jacobians are too big

Suppose a layer maps 4,096 numbers to 4,096 numbers. Its Jacobian is $4096 \times 4096 \approx 16.8$ million entries. Fine for one layer. But now consider the Jacobian of the *loss with respect to all parameters* of a model with 10^9 parameters. And the Jacobian of an entire network's output (say, 50,000 vocabulary logits) with respect to 10^9 parameters would have 5×10^{13} entries. That is hundreds of terabytes. **We can never build the full Jacobian for real models.**

But here's the key insight: **we almost never need the Jacobian itself.** We only need what it *does to a vector*. There are exactly two useful things we can do:

1. Multiply J by a vector on the **right**: $J\mathbf{v}$. (JVP)
2. Multiply J by a vector on the **left**: $\mathbf{u}^\top J$. (VJP)

And remarkably, both can be computed at a cost comparable to evaluating the function once, **without ever forming** J.

---

8. JVP: pushing a nudge forward (the Jacobian-Vector Product)

8.1 Definition

$$\mathrm{JVP}(f, \mathbf{x}, \mathbf{v}) = J_f(\mathbf{x})\,\mathbf{v}$$

where $\mathbf{v} \in \mathbb{R}^n$ (same size as the input). The result lives in $\mathbb{R}^m$ (same size as the output).

8.2 What it means

By Section 5.2: **the JVP is the output nudge caused by an input nudge of direction $\mathbf{v}$.** It answers:

> "If I move my inputs a tiny bit in the direction $\mathbf{v}$, how do all my outputs move?"

Equivalently, it is the **directional derivative** of f in direction $\mathbf{v}$:

$$J\mathbf{v} = \lim_{\varepsilon \to 0} \frac{f(\mathbf{x} + \varepsilon \mathbf{v}) - f(\mathbf{x})}{\varepsilon}$$

This also tells you what the JVP *is not*: it isn't a number you could guess without the function; it's what you'd get by (infinitesimally) wiggling the inputs and watching the outputs. In fact, you could estimate it with **finite differences**: pick a tiny ε like 10^{-5}, evaluate f twice, subtract, divide. Automatic differentiation does the same thing *exactly* (no tiny-ε rounding errors).

8.3 Example

Using the earlier function at $\mathbf{x} = (1, 2)$ with $\mathbf{v} = (1, 0.5)$:

$$J\mathbf{v} = \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ 0.5403 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 3 \\ 0.5403 \end{bmatrix}$$

Interpretation: moving the input along $(1, 0.5)$ changes output 1 at rate 2.5, output 2 at rate 3, output 3 at rate 0.54.

8.4 How it's computed without building J ("forward mode")

Walk through the computation *as you evaluate* f, carrying along, with each intermediate value, how that value changes under the nudge.

Step 0: What do the dots ($\dot{x}_1$, $\dot{a}$, ...) mean?

Imagine a **slider** t that walks you away from the starting point $(1, 2)$ in the direction $\mathbf{v} = (1, 0.5)$:

$$x_1(t) = 1 + 1 \cdot t, \qquad x_2(t) = 2 + 0.5 \cdot t$$

At $t = 0$ you are exactly at $(1, 2)$. As t grows, both inputs move together: x_1 at speed 1, and x_2 at speed 0.5. (The slider t is only a bookkeeping device; it is not part of f.)

A dot means **"derivative with respect to the slider** t**"**, that is, *how fast this quantity changes as you slide along* $\mathbf{v}$:

$$\dot{x}_1 = \frac{dx_1}{dt} = 1, \qquad \dot{x}_2 = \frac{dx_2}{dt} = 0.5$$

So for the inputs, $\dot{x}_1$ **and** $\dot{x}_2$ **are simply the entries of** $\mathbf{v}$: $\dot{x}_1 = v_1$ and $\dot{x}_2 = v_2$. They are the "speeds" of the inputs.

Now the key point: once the inputs depend on t, **every quantity computed from them depends on** t **too**. So every intermediate value a gets its own dotted partner $\dot{a} = da/dt$, which is its **tangent**: the rate at which a changes along $\mathbf{v}$. Forward mode computes each value and its tangent side by side, using the ordinary derivative rules (sum rule, product rule, chain rule) one operation at a time.

For example, $f_1 = x_1 x_2$. Both factors change with t, so the product rule gives

$$\dot{f}_1 = \frac{d}{dt}(x_1 x_2) = \dot{x}_1\, x_2 + x_1\, \dot{x}_2 = (1)(2) + (1)(0.5) = 2.5$$

The full walkthrough

For our function with $x_1 = 1, x_2 = 2$ and $\mathbf{v} = (1, 0.5)$, so $\dot{x}_1 = 1$, $\dot{x}_2 = 0.5$:

Operation	Value	Tangent (via ordinary calculus rules)
$f_1 = x_1 x_2$	2	$\dot{x}_1 x_2 + x_1 \dot{x}_2 = 1 \cdot 2 + 1 \cdot 0.5 = 2.5$
$q = x_2^2$	4	$2 x_2\, \dot{x}_2 = 4 \cdot 0.5 = 2$
$f_2 = x_1 + q$	5	$\dot{x}_1 + \dot{q} = 1 + 2 = 3$

Operation	Value	Tangent (via ordinary calculus rules)
$f_3 = \sin x_1$	0.8415	$\cos x_1 \, \dot{x}_1 = 0.5403$

Result: $(\dot{f}_1, \dot{f}_2, \dot{f}_3) = (2.5, 3, 0.5403)$, matching $J\mathbf{v}$.

Why must these match $J\mathbf{v}$? Along the path $\mathbf{x}(t)$, the chain rule says

$$\dot{f}_i = \frac{df_i}{dt} = \frac{\partial f_i}{\partial x_1}\dot{x}_1 + \frac{\partial f_i}{\partial x_2}\dot{x}_2 = \sum_j J_{ij}\, v_j = (J\mathbf{v})_i$$

So the vector of output tangents $(\dot{f}_1, \dot{f}_2, \dot{f}_3)$ *is* $J\mathbf{v}$. "The rate of change along the path" and "the output nudge per unit of input nudge" are the same thing: a tiny step ε along $\mathbf{v}$ changes the outputs by about $\varepsilon \cdot J\mathbf{v}$.

Symbol	Meaning
$\mathbf{v} = (1, 0.5)$	the direction you chose to move the inputs
$\dot{x}_1,\ \dot{x}_2$	how fast each input moves along that path (just v_1, v_2)
$\dot{a}$ (any intermediate value)	how fast that value changes along the path
$(\dot{f}_1, \dot{f}_2, \dot{f}_3)$	the JVP, $J\mathbf{v}$

We never wrote down J. We only applied the simple, local derivative rules to each operation, alongside the normal calculation. The extra cost is a small constant factor (about 2 to 3 times the cost of evaluating f).

This is the idea behind **dual numbers**: numbers of the form $a + \dot{a}\,\varepsilon$ where $\varepsilon^2 = 0$. Do arithmetic on them as usual and the ε part automatically carries the derivative.

8.5 What one JVP gives you, and what it costs

One JVP gives you **one column's worth** of information (the combined effect of one input direction). To recover the *entire* Jacobian with JVPs you'd need n of them: one for each basis vector $\mathbf{e}_1, \ldots, \mathbf{e}_n$ (since $J\mathbf{e}_j$ is just column j).

8.6 Matrix route vs. forward-mode route: a three-layer example

You may notice that for our small function, "multiply the Jacobian by $\mathbf{v}$" (Section 8.3) and "carry tangents through a table" (Section 8.4) do exactly the same arithmetic. That is true for a tiny function. The difference only becomes visible when the function is a **composition** of several steps, as a neural network is:

$$f = f_3 \circ f_2 \circ f_1, \qquad \mathbf{x} \xrightarrow{f_1} \mathbf{y} \xrightarrow{f_2} \mathbf{h} \xrightarrow{f_3} \mathbf{o}$$

Let's use a mini network with three layers:

- f_1: linear, $\mathbf{y} = W_1\mathbf{x}$ with $W_1 = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}$
- f_2: ReLU, $\mathbf{h} = \mathrm{ReLU}(\mathbf{y})$ (applied entry by entry)
- f_3: linear, $\mathbf{o} = W_3\mathbf{h}$ with $W_3 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}$ (this one has 3 outputs)

We evaluate at $\mathbf{x} = (1, 2)$ and nudge along $\mathbf{v} = (1,\ 0.5)$.

Forward pass (the values):

$$\mathbf{y} = W_1\mathbf{x} = (1 - 2,\ 2 + 2) = (-1,\ 4), \qquad \mathbf{h} = \mathrm{ReLU}(\mathbf{y}) = (0,\ 4), \qquad \mathbf{o} = W_3\mathbf{h} = (0,\ 4,\ 12)$$

Route A: the matrix route (build J first)

Write down the Jacobian of each layer:

$$J_1 = W_1 = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}, \qquad J_2 = \mathrm{diag}\Big([\,y > 0\,]\Big) = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \qquad J_3 = W_3 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}$$

(The ReLU Jacobian is diagonal: it has a 1 where the unit was active, $y_i > 0$, and a 0 where it was not.)

Chain rule: $J = J_3 J_2 J_1$. Multiply the matrices together first:

$$J_2 J_1 = \begin{bmatrix} 0 & 0 \\ 2 & 1 \end{bmatrix}, \qquad J = J_3(J_2 J_1) = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix}$$

Finally apply it to the nudge:

$$J\mathbf{v} = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 2.5 \\ 7.5 \end{bmatrix}$$

Route B: the forward-mode route (carry the tangent through)

Never build any J_i. Start with the tangent $\dot{\mathbf{x}} = \mathbf{v} = (1, 0.5)$ and push it through each layer using only that layer's own local rule:

Layer	Local rule for the tangent	Computation	Result
f_1 (linear)	$\dot{\mathbf{y}} = W_1\dot{\mathbf{x}}$	$(1 \cdot 1 - 1 \cdot 0.5,\ \ 2 \cdot 1 + 1 \cdot 0.5)$	$\dot{\mathbf{y}} = (0.5,\ 2.5)$

Layer	Local rule for the tangent	Computation	Result
f_2 (ReLU)	$\dot{\mathbf{h}} = [\,y > 0\,] \odot \dot{\mathbf{y}}$	$(0, 1) \odot (0.5,\ 2.5)$	$\dot{\mathbf{h}} = (0,\ 2.5)$
f_3 (linear)	$\dot{\mathbf{o}} = W_3 \dot{\mathbf{h}}$	$(1 \cdot 0,\ \ 2 \cdot 0 + 1 \cdot 2.5,\ \ 3 \cdot 2.5)$	$\dot{\mathbf{o}} = (0,\ 2.5,\ 7.5)$

Same answer: $J\mathbf{v} = (0,\ 2.5,\ 7.5)$.

What is actually different?

1. **Route A multiplied two matrices together** ($J_2 J_1$, then J_3 times that). **Route B only ever multiplied a matrix by a vector** (or, for ReLU, just a vector by a vector, entry by entry).
2. **Route A needed all three Jacobians to exist as matrices.** In particular it built a diagonal matrix for ReLU. **Route B only used each layer's local rule**, and for ReLU that is just "zero out the entries that were inactive." The diagonal matrix was never created.
3. **Route A needed the Jacobian of the *whole* function** to be assembled before the nudge was used. **Route B pushed the nudge through as the function was being evaluated**, layer by layer, so it can run right alongside the forward pass.

An honest note: if in Route A you *also* chose to multiply right to left, i.e. $J_3\Big(J_2(J_1\mathbf{v})\Big)$, you would do the very same arithmetic as Route B. That is precisely the point: **forward mode is "multiply right to left, one vector at a time," with each J_i replaced by its cheap local rule so that it never has to be stored.**

Counting the operations in our example

Both orders give the same answer, but they do not do the same amount of work. Let's count multiplications. Multiplying an $(a \times b)$ matrix by a $(b \times c)$ matrix takes $a \cdot b \cdot c$ multiplications, and a matrix times a vector is the special case $c = 1$. Here J_1 is 2×2, J_2 is 2×2 (the ReLU diagonal, treated as an ordinary dense matrix), J_3 is 3×2, and $\mathbf{v}$ has 2 entries.

Order A: $(J_3 J_2 J_1)\,\mathbf{v}$ (multiply the matrices together first)

Step	Shapes	Multiplications
$J_2 J_1$	$(2 \times 2)(2 \times 2)$	$2 \cdot 2 \cdot 2 = 8$
$J_3(J_2 J_1)$	$(3 \times 2)(2 \times 2)$	$3 \cdot 2 \cdot 2 = 12$
$J\mathbf{v}$	$(3 \times 2)(2 \times 1)$	$3 \cdot 2 \cdot 1 = 6$
Total		**26**

Order B: $J_3\Big(J_2(J_1\mathbf{v})\Big)$ (push the vector through one matrix at a time)

Step	Shapes	Multiplications
$J_1\mathbf{v}$	$(2 \times 2)(2 \times 1)$	$2 \cdot 2 \cdot 1 = 4$
$J_2(J_1\mathbf{v})$	$(2 \times 2)(2 \times 1)$	$2 \cdot 2 \cdot 1 = 4$
$J_3(\cdots)$	$(3 \times 2)(2 \times 1)$	$3 \cdot 2 \cdot 1 = 6$
Total		**14**

The same answer costs 26 multiplications in one order and 14 in the other. If we also use ReLU's local rule (an entry-by-entry multiply instead of a dense 2×2 product), the middle step drops from 4 to 2 and the total is 12. This is the actual forward-mode computation from the table in Route B.

Why the gap exists: Order A contains two matrix-by-matrix products, which are the expensive operations. Order B contains only matrix-by-vector products. The final step of Order A, $J\mathbf{v}$, is also matrix-by-vector, which is why the 6 appears in both tables.

How the gap grows. Suppose every layer has width d (our example has a 3-output last layer, so its counts differ slightly):

	Order A	Order B
matrix-by-matrix products	2, each costing d^3	0
matrix-by-vector products	1, costing d^2	3, each costing d^2
Total	$2d^3 + d^2$	$3d^2$

At $d = 2$ these are close (about 20 vs 12), which is why our example makes the two orders look nearly the same. At $d = 4096$:

- Order A: about 1.4×10^{11} multiplications.
- Order B: about 5×10^7 multiplications.

That is a gap of roughly 2,700 times, and it keeps growing in proportion to d. With more layers, Order A adds another d^3 matrix-by-matrix product for every extra layer, while Order B adds only another d^2.

> **So the reordering saves time, not just memory.** The saving comes from avoiding matrix-by-matrix products entirely, and that is a separate benefit from never storing the J_i matrices.

Why it matters at real sizes

Suppose each layer has width d (inputs and outputs of size d) and the network has L layers.

	Route A (build matrices, multiply them)	**Route B (forward-mode tangents)**
Multiplying two Jacobians	about d^3 operations each time	never done
Applying a linear layer	(part of the matrix product)	d^2 (matrix times vector)
Applying an elementwise layer (e.g. ReLU)	stores a $d \times d$ diagonal matrix	d (entry-wise multiply)
Total time	about $L\,d^3$	about $L\,d^2$
Extra memory	d^2 per Jacobian	d per tangent

For $d = 4096$ the time difference is a factor of about 4000, and each ReLU layer would otherwise cost a 16.8-million-entry matrix to hold something that is really a single list of 4096 zeros and ones.

So the two routes are the same mathematics, but Route B is the one that can be run on huge networks, and it is the one automatic differentiation uses.

> **Takeaway:** JVP = "forward propagation of a nudge." Cost ≈ one function evaluation. Gives you $J\mathbf{v}$, an *output-sized* vector.

---

9. VJP: pulling a sensitivity backward (the Vector-Jacobian Product)

9.1 Definition

$$\mathrm{VJP}(f, \mathbf{x}, \mathbf{u}) = \mathbf{u}^\top J_f(\mathbf{x})$$

where $\mathbf{u} \in \mathbb{R}^m$ (same size as the *output*). The result is a row vector of length n (same size as the *input*). Many libraries return it as a column vector, i.e. $J^\top \mathbf{u}$. These are the same numbers in a different layout.

Notice the sizes flip compared to the JVP: **the JVP takes an input-sized vector and returns an output-sized one. The VJP takes an output-sized vector and returns an input-sized one.**

9.2 What it means

Think of $\mathbf{u}$ as a set of **weights saying how much you care about each output**. Define a single scalar "score":

$$s = \mathbf{u}^\top f(\mathbf{x}) = u_1 f_1 + u_2 f_2 + \cdots + u_m f_m$$

Differentiate s with respect to the inputs. By the chain rule, you get exactly $\mathbf{u}^\top J$. So:

> **The VJP is the gradient (with respect to the inputs) of the weighted sum of outputs, where $\mathbf{u}$ provides the weights.**
>
> It answers: *"If I care about the outputs in proportion to $\mathbf{u}$, how sensitive is that to each input?"*

How to read the result. The VJP is a vector with one entry per input. Entry j means:

$$\text{nudge } x_j \text{ by a tiny } \varepsilon \implies s \text{ changes by about } \varepsilon \cdot (\mathbf{u}^\top J)_j$$

- A **large** entry means the score is very sensitive to that input.
- A **negative** entry means increasing that input *decreases* the score.
- A **zero** entry means that input has (locally) no influence on the score.

JVP and VJP ask opposite questions:

	Question it answers	You choose	You get
JVP	"I move the inputs this way. What happens to *every output*?"	a direction $\mathbf{v}$ for the inputs	the change in each output
VJP	"I care about the outputs this much. Which *inputs* are responsible?"	weights $\mathbf{u}$ for the outputs	the sensitivity of the score to each input

Note that the JVP does not need a score. It reports what happens to all m outputs for one chosen input direction. The VJP *collapses* the m outputs into one score first, and then reports what every input does to that single number. That collapse is what makes it so cheap when many inputs feed one loss.

9.3 Example

Same function, $\mathbf{x} = (1, 2)$.

- With $\mathbf{u} = (1, 0, 0)$: you only care about output 1. $\mathbf{u}^\top J$ = row 1 of J = $(2, 1)$. (That's just ∇f_1.)
- With $\mathbf{u} = (1, 1, 1)$: you care about the sum of outputs. $\mathbf{u}^\top J$ = sum of the rows = $(2 + 1 + 0.5403,\ 1 + 4 + 0) = (3.5403,\ 5)$.

Check the $\mathbf{u} = (1, 1, 1)$ case with actual numbers. The score is $s = f_1 + f_2 + f_3$, and at $(1, 2)$ it equals $2 + 5 + 0.841471 = 7.841471$.

- Nudge x_1 from 1 to 1.01: $f = (2.02,\ 5.01,\ 0.846832)$, so $s = 7.876832$. The change is 0.035361, which is about 3.536×0.01. This matches the first VJP entry 3.5403. ✓
- Nudge x_2 from 2 to 2.01: $f = (2.01,\ 5.0401,\ 0.841471)$, so $s = 7.891571$. The change is 0.05010, which is about 5.01×0.01. This matches the second VJP entry 5. ✓

(The small mismatches are the usual curvature leftovers, which shrink as the nudge shrinks.)

Notice: choosing $\mathbf{u}$ as a basis vector $\mathbf{e}_i$ extracts *row* i of the Jacobian. So to recover the entire Jacobian with VJPs, you need m of them (one per output).

9.4 How it's computed without building J ("reverse mode")

Reverse mode works in two phases:

1. **Forward pass:** evaluate f normally, *recording* the intermediate values (they'll be needed).
2. **Backward pass:** start with $\mathbf{u}$ at the output and flow backward through each operation. At each operation, apply that operation's local "transposed" derivative rule.

For each intermediate value a, we track its **adjoint** $\bar{a}$ (read "a-bar"): *how much the final weighted score s changes per unit change in a*. That is, $\bar{a} = \partial s / \partial a$.

The recipe at a single operation $\mathbf{b} = g(\mathbf{a})$: if you know $\bar{\mathbf{b}}$ (sensitivity of the score to $\mathbf{b}$), then

$$\bar{\mathbf{a}} = J_g(\mathbf{a})^\top \bar{\mathbf{b}}$$

and if a value is used in multiple places, **its adjoints add up** (that's the multivariable chain rule: contributions from every path are summed).

Let's run it on our function with $\mathbf{u} = (1, 1, 1)$ at $\mathbf{x} = (1, 2)$. Start: $\bar{f}_1 = \bar{f}_2 = \bar{f}_3 = 1$.

Step (reverse order)	Rule	Result
$f_3 = \sin x_1$	$\bar{x}_1 \mathrel{+}= \cos(x_1)\, \bar{f}_3$	$\bar{x}_1 = 0.5403$
$f_2 = x_1 + q$	$\bar{x}_1 \mathrel{+}= \bar{f}_2,\ \bar{q} = \bar{f}_2$	$\bar{x}_1 = 1.5403$, $\bar{q} = 1$
$q = x_2^2$	$\bar{x}_2 \mathrel{+}= 2x_2\, \bar{q}$	$\bar{x}_2 = 4$
$f_1 = x_1 x_2$	$\bar{x}_1 \mathrel{+}= x_2 \bar{f}_1,\ \bar{x}_2 \mathrel{+}= x_1 \bar{f}_1$	$\bar{x}_1 = 3.5403$, $\bar{x}_2 = 5$

Final: $(\bar{x}_1, \bar{x}_2) = (3.5403, 5)$, matching $\mathbf{u}^\top J$. Again: no Jacobian was ever built.

9.5 What do the bars ($\bar{x}_1$, $\bar{a}$, ...) mean?

This is the mirror image of the dots in Section 8.4.

The dot ($\dot{a}$) answers: *"if the inputs move along $\mathbf{v}$, how fast does a change?"* It looks **back toward the inputs**: it depends on everything that came *before* a.

The bar ($\bar{a}$) answers: *"if I wiggle a by a tiny amount, and let everything* after *a react, how much does the final score s change?"* It looks **ahead toward the output**: it depends on everything that comes *after* a.

$$\bar{a} = \frac{\partial s}{\partial a}$$

Two consequences follow:

1. **The starting point is known immediately.** At the very end, the score is $s = u_1 f_1 + \cdots + u_m f_m$, so $\partial s / \partial f_i = u_i$. That is, $\bar{f}_i = u_i$. The backward pass starts from the weights you chose.
2. **To get $\bar{a}$ you need to know everything that happens after a**, which is why the pass has to run from the output back to the input. In forward mode you only need what happened *before* a value, so it can run alongside the normal evaluation.

Why adjoints add up (fan-out). In our function, x_1 is used in *three* places: in $f_1 = x_1 x_2$, in $f_2 = x_1 + q$, and in $f_3 = \sin x_1$. Wiggling x_1 disturbs all three outputs, and s feels all three disturbances. So the effects add:

Path from x_1 to s	**Contribution to $\bar{x}_1$**
through $f_1 = x_1 x_2$	$x_2 \cdot \bar{f}_1 = 2 \cdot 1 = 2$
through $f_2 = x_1 + q$	$1 \cdot \bar{f}_2 = 1$
through $f_3 = \sin x_1$	$\cos(x_1) \cdot \bar{f}_3 = 0.5403$
Total	$\bar{x}_1 = 3.5403$ ✓

Likewise x_2 is used in f_1 (contribution $x_1 \bar{f}_1 = 1$) and in $q = x_2^2$ (contribution $2x_2\,\bar{q} = 4$), giving $\bar{x}_2 = 5$ ✓. That is exactly why the table in 9.4 uses "$+=$": each use of a value *adds* its contribution into that value's adjoint.

9.6 Matrix route vs. reverse-mode route: the same three-layer example

Let's reuse the mini network from Section 8.6 so you can compare JVP and VJP on identical ground:

$$\mathbf{x} \xrightarrow{f_1:\ \mathbf{y}=W_1\mathbf{x}} \mathbf{y} \xrightarrow{f_2:\ \mathbf{h}=\mathrm{ReLU}(\mathbf{y})} \mathbf{h} \xrightarrow{f_3:\ \mathbf{o}=W_3\mathbf{h}} \mathbf{o}$$

with $W_1 = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}$, $W_3 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}$, input $\mathbf{x} = (1, 2)$, so that $\mathbf{y} = (-1, 4)$, $\mathbf{h} = (0, 4)$, $\mathbf{o} = (0, 4, 12)$. We choose $\mathbf{u} = (1, 1, 1)$, so the score is $s = o_1 + o_2 + o_3 = 16$.

Route A: the matrix route (build J first)

From Section 8.6 we already have the full Jacobian:

$$J = J_3 J_2 J_1 = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix}$$

Then:

$$\mathbf{u}^\top J = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix} = (8,\ 4)$$

Route B: the reverse-mode route (carry the cotangent backward)

Never build any J_i. Start with the cotangent $\bar{\mathbf{o}} = \mathbf{u} = (1, 1, 1)$ and pull it backward through each layer using only that layer's local rule. Recall from 9.4 that the rule is $\bar{\mathbf{a}} = J_g^\top \bar{\mathbf{b}}$, which for each layer type gives:

Layer (in reverse order)	Local rule for the cotangent	Computation	Result
f_3 (linear)	$\bar{\mathbf{h}} = W_3^\top \bar{\mathbf{o}}$	$(1 \cdot 1 + 2 \cdot 1 + 0 \cdot 1,\ \ 0 \cdot 1 + 1 \cdot 1 + 3 \cdot 1)$	$\bar{\mathbf{h}} = (3,\ 4)$
f_2 (ReLU)	$\bar{\mathbf{y}} = [\, y > 0 \,] \odot \bar{\mathbf{h}}$	$(0, 1) \odot (3,\ 4)$	$\bar{\mathbf{y}} = (0,\ 4)$
f_1 (linear)	$\bar{\mathbf{x}} = W_1^\top \bar{\mathbf{y}}$	$\begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix}$	$\bar{\mathbf{x}} = (8,\ 4)$

Same answer: $\mathbf{u}^\top J = (8, 4)$.

Bonus: the parameter gradients come out for free. At each linear layer, the cotangent that arrives is *also* all you need for that layer's weights (Section 13.3: $\bar{W} = \bar{\mathbf{o}}\mathbf{ut}\ \mathbf{in}^\top$):

$$\bar{W}_3 = \bar{\mathbf{o}}\, \mathbf{h}^\top = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 4 \end{bmatrix} = \begin{bmatrix} 0 & 4 \\ 0 & 4 \\ 0 & 4 \end{bmatrix}, \qquad \bar{W}_1 = \bar{\mathbf{y}}\, \mathbf{x}^\top = \begin{bmatrix} 0 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 4 & 8 \end{bmatrix}$$

One backward pass delivered the sensitivity of s to the input *and* to every weight. This is what training uses.

What is actually different?

1. **Route A multiplied matrices together** ($J_2 J_1$, then J_3 times that). **Route B only multiplied a vector by a matrix** (or, for ReLU, a vector by a vector, entry by entry).
2. **Route A needed every Jacobian to exist as a matrix**, including a diagonal one for ReLU. **Route B used only local rules.** For ReLU the local rule is "zero out the entries that were inactive."
3. **Direction.** In Route B the vector meets J_3 first, then J_2, then J_1. That is the *opposite* order from the forward pass, and that is why it is called the backward pass.
4. **Route B needs the forward pass's values.** To apply the ReLU rule we need the mask $[y > 0]$, which comes from $\mathbf{y} = (-1, 4)$ computed earlier. Forward mode did not have this requirement, because it computed the mask on the fly and could throw it away. In reverse mode the forward pass must happen *first*, and its intermediate values must be kept until the backward pass uses them. This is the memory cost of reverse mode (Section 12).

As in Section 8.6, there is an honest note: if Route A multiplied left to right, $\Big((\mathbf{u}^\top J_3) J_2\Big) J_1$, it would do exactly the same arithmetic as Route B. Reverse mode is "multiply left to right, one vector at a time," with each J_i replaced by its cheap local rule so it never has to be stored.

Counting the operations in our example

A $(1 \times a)$ vector times an $(a \times b)$ matrix takes $a \cdot b$ multiplications.

Order A: $\mathbf{u}^\top (J_3 J_2 J_1)$ (multiply the matrices first)

Step	Shapes	Multiplications
$J_2 J_1$	$(2 \times 2)(2 \times 2)$	8
$J_3 (J_2 J_1)$	$(3 \times 2)(2 \times 2)$	12
$\mathbf{u}^\top J$	$(1 \times 3)(3 \times 2)$	6
Total		**26**

Order B: $\Big((\mathbf{u}^\top J_3) J_2\Big) J_1$ (push the vector through one matrix at a time)

Step	Shapes	Multiplications
$\mathbf{u}^\top J_3$	$(1 \times 3)(3 \times 2)$	6
$(\cdot)\, J_2$	$(1 \times 2)(2 \times 2)$	4
$(\cdot)\, J_1$	$(1 \times 2)(2 \times 2)$	4
Total		**14**

(With ReLU's local rule, the middle step becomes an entry-by-entry multiply costing 2, so the total is 12.)

These are the same counts as in the JVP case, for the same reason: the expensive part of Order A is the two matrix-by-matrix products. With every layer of width d, Order A costs about $2d^3 + d^2$ and Order B about $3d^2$, so the same reasoning as in Section 8.6 applies. At $d = 4096$ the gap is again about 2,700 times.

9.7 What one VJP gives you, and what it costs

This mirrors Section 8.5. Take the full Jacobian of our three-layer network:

$$J = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix} \qquad (3 \text{ outputs},\ 2 \text{ inputs})$$

A JVP reads one column. With $\mathbf{v} = \mathbf{e}_1$ you get $(0, 2, 6)$, and with $\mathbf{v} = \mathbf{e}_2$ you get $(0, 1, 3)$. The full J needs $n = 2$ JVPs.

A VJP reads one row. With $\mathbf{u} = \mathbf{e}_1$ you get $(0, 0)$, with $\mathbf{u} = \mathbf{e}_2$ you get $(2, 1)$, and with $\mathbf{u} = \mathbf{e}_3$ you get $(6, 3)$. The full J needs $m = 3$ VJPs.

Here the two counts are similar, so neither mode wins. Now look at the case that matters for training. A loss has $m = 1$ output:

	Inputs n	**Outputs** m	**JVPs for the full gradient**	**VJPs for the full gradient**
Our small network	2	3	2	3
A loss over 10^9 parameters	10^9	1	10^9	**1**

The Jacobian of a loss is a single row, and the VJP reads a whole row in one pass. Each backward pass costs a small multiple (about 2 to 3 times) of one forward pass, regardless of how many parameters there are.

> **Takeaway so far:** a JVP reads one *column* of J per pass, a VJP reads one *row* of J per pass. Pick the mode that needs fewer passes.

9.8 The magic property

For a **scalar-output** function ($m = 1$), choose $u = 1$. A *single* backward pass gives you the **entire gradient** with respect to *all n inputs*, no matter how huge n is, at a cost of only a few times the forward pass.

That is the miracle behind training billion-parameter models.

> **Takeaway:** VJP = "backward propagation of a sensitivity." Cost ≈ a small multiple of one function evaluation. Gives you $\mathbf{u}^\top J$, an *input-sized* vector.

---

10. JVP vs. VJP at a glance

This section pulls Sections 8 and 9 together. If you only re-read one part of the guide later, make it this one.

In one sentence each

- **JVP** ($J\mathbf{v}$): *"I nudge the inputs in direction $\mathbf{v}$. How does every output move?"* It pushes a nudge **forward**.
- **VJP** ($\mathbf{u}^\top J$): *"I weight the outputs by $\mathbf{u}$ to form one score. How sensitive is that score to every input?"* It pulls a sensitivity **backward**.

Side-by-side comparison

For $f : \mathbb{R}^n \to \mathbb{R}^m$ with Jacobian J of shape $m \times n$:

	JVP (forward mode)	**VJP (reverse mode)**
Formula	$J\mathbf{v}$	$\mathbf{u}^\top J$ (or $J^\top \mathbf{u}$)
You supply	$\mathbf{v} \in \mathbb{R}^n$ (input-sized)	$\mathbf{u} \in \mathbb{R}^m$ (output-sized)
You get	a vector in $\mathbb{R}^m$ (output-sized)	a vector in $\mathbb{R}^n$ (input-sized)
What the input vector means	a *direction* to move the inputs	*weights* for combining the outputs into a score
What the result means	change in each output	sensitivity of the score to each input
Name of the vectors carried	**tangents** ($\dot{a}$)	**cotangents / adjoints** ($\bar{a}$)
Direction of the computation	same as the forward pass (input $\to$ output)	opposite to the forward pass (output $\to$ input)
Local rule per layer	multiply the tangent by J_i	multiply the cotangent by $J_i^\top$
Linear layer W	$\dot{\mathbf{out}} = W\,\dot{\mathbf{in}}$	$\bar{\mathbf{in}} = W^\top \bar{\mathbf{out}}$
ReLU	$\dot{\mathbf{h}} = \text{mask} \odot \dot{\mathbf{y}}$	$\bar{\mathbf{y}} = \text{mask} \odot \bar{\mathbf{h}}$
One pass reads	one **column** of J (with $\mathbf{v} = \mathbf{e}_j$)	one **row** of J (with $\mathbf{u} = \mathbf{e}_i$)

	JVP (forward mode)	VJP (reverse mode)
Passes for the full J	n	m
Cost of one pass	about 2 to 3 times one evaluation	about 2 to 3 times one evaluation
Must store forward values?	no (computed on the fly)	**yes** (the backward pass needs them)
Best when	few inputs, many outputs ($n \ll m$)	many inputs, few outputs ($n \gg m$), such as a loss
In machine learning	directional derivatives, curvature products, sensitivity analysis	**backpropagation** and every gradient used in training

The same network, both ways

Using the three-layer network from Sections 8.6 and 9.6 at $\mathbf{x} = (1, 2)$, where $J = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix}$:

```
Forward pass (values):   x=(1,2) --f1--> y=(-1,4) --f2--> h=(0,4) --f3--> o=
(0,4,12)

JVP, v=(1, 0.5):         (1, 0.5) --W1--> (0.5, 2.5) --mask--> (0, 2.5) --
W3--> (0, 2.5, 7.5)
                          tangents flow WITH the forward pass
result = J v

VJP, u=(1, 1, 1):        (8, 4) <--W1^T-- (0, 4) <--mask-- (3, 4) <--W3^T--
(1, 1, 1)
                          cotangents flow AGAINST the forward pass
result = u^T J
```

	Input vector	Result	Reading
JVP	$\mathbf{v} = (1,\ 0.5)$	$J\mathbf{v} = (0,\ 2.5,\ 7.5)$	the outputs move at rates $0,\ 2.5,\ 7.5$ along that nudge
VJP	$\mathbf{u} = (1,\ 1,\ 1)$	$\mathbf{u}^\top J = (8,\ 4)$	the score $s = o_1 + o_2 + o_3$ changes by 8ε per ε nudge of x_1 and 4ε per ε nudge of x_2

The link between them

Both answer the same underlying question from opposite ends, *"how much does the score change when I nudge the inputs along $\mathbf{v}$?"*:

$$\mathbf{u}^\top (J\mathbf{v}) \;=\; (\mathbf{u}^\top J)\,\mathbf{v}$$

With our numbers: $(1,1,1)\cdot(0,2.5,7.5) = 10$ and $(8,4)\cdot(1,0.5) = 10$. ✓ The JVP finds it by pushing one specific nudge forward. The VJP finds the sensitivity to *every* input at once, which you can then dot with any nudge you like.

How to choose

1. **Count the sizes.** Is n (inputs) much larger than m (outputs)? Use the VJP. Is m much larger than n? Use the JVP.
2. **Is the output a single number, like a loss?** Use the VJP. One backward pass gives the gradient with respect to *all* inputs and parameters.
3. **Do you want the effect of a specific nudge on every output?** Use the JVP.
4. **Is memory tight, and the function has few inputs?** Consider the JVP, since it doesn't have to store the forward pass.

Four things to remember

- **Same J, two directions.** The JVP multiplies J on the right (input-sized vector in). The VJP multiplies J on the left (output-sized vector in).
- **Neither ever builds J.** Both push a vector through the layers using each layer's local rule, so every step is a cheap vector operation.
- **Column vs. row.** A JVP reads a column and a VJP reads a row. A loss has one row, so reverse mode wins training.
- **Backpropagation is the VJP,** applied layer by layer from the loss back to the weights.

---

11. Tangents vs. cotangents, demystified

You'll see these words in JAX documentation and papers. They sound scary but map directly onto what we just did.

11.1 Tangent vectors (the "nudge" side)

A **tangent vector** at a point is a *direction in which you can move* from that point, i.e. a perturbation of the inputs. In our notation: $\boldsymbol{\delta}$ or $\mathbf{v}$ (inputs) and $\dot{a}$ (intermediates). Recall from Section 8.4 that $\dot{a}$ means "how fast a changes as you slide along the direction $\mathbf{v}$", and that for the inputs $\dot{x}_j$ is just v_j. The JVP takes a tangent in and yields a tangent out. This is why JVP is sometimes called "forward-mode" or the "pushforward": **it pushes tangents forward** through f.

Tangents live in the **same space as the quantity they perturb.** The tangent of a d-dimensional vector is a d-dimensional vector.

11.2 Cotangent vectors (the "sensitivity" side)

A **cotangent vector** (or *covector*) is a *measuring device*: a thing that takes a nudge and returns a single number (a linear function of the nudge). The gradient is the perfect example. Given a nudge $\boldsymbol{\delta}$, it returns the resulting change in f: $\nabla f^\top \boldsymbol{\delta}$.

In our notation, $\mathbf{u}$ (a weighting over outputs) and the adjoints $\bar{a}$ are cotangents. The VJP takes a cotangent over the *output* and returns a cotangent over the *input*. This is the "pullback": **it pulls cotangents backward** through f.

11.3 Why bother with two names if both are just vectors?

In ordinary $\mathbb{R}^n$ they're both lists of n numbers, so the distinction seems pedantic. But they **behave differently**:

- Tangents go **forward**; multiplied by J.
- Cotangents go **backward**; multiplied by $J^\top$.
- The pairing between them is a number: $\mathbf{u}^\top(J\mathbf{v})$, which is the same as $(\mathbf{u}^\top J)\mathbf{v}$. This single identity is the entire logic behind why both modes agree:

$$\underbrace{\mathbf{u}^\top}_{\text{cotangent}} \underbrace{(J\mathbf{v})}_{\text{pushed-forward tangent}} = \underbrace{(\mathbf{u}^\top J)}_{\text{pulled-back cotangent}} \underbrace{\mathbf{v}}_{\text{tangent}}$$

Both sides answer: *"how much does my weighted output change when I nudge the input along $\mathbf{v}$?"*, just computed from opposite ends.

Keeping the two roles separate prevents bugs in advanced settings (e.g. non-Euclidean spaces, or when sizes happen to coincide and you mix them up).

11.4 The four objects, with numbers

Take the three-layer network from Sections 8.6 and 9.6 at $\mathbf{x} = (1, 2)$, with $J = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix}$. If we choose a nudge $\mathbf{v}$ and a set of output weights $\mathbf{u}$, exactly four vectors appear, and each one has a name:

Object	Value	Name	Lives at
$\mathbf{v} = (1,\ 0.5)$	your chosen nudge	**input tangent**	the input
$J\mathbf{v}$ $= (0,\ 2.5,\ 7.5)$	how fast each output changes along $\mathbf{v}$	**output tangent** (the pushed-forward $\mathbf{v}$)	the output
$\mathbf{u} = (1,\ 1,\ 1)$	weights defining the score s $= o_1 + o_2 + o_3$	**output cotangent**	the output

Object	Value	Name	Lives at
$\mathbf{u}^\top J = (8,\ 4)$	sensitivity of the score to each input	**input cotangent** (the pulled-back $\mathbf{u}$)	the input

The two operations connect them:

- The **JVP** carries a tangent across the function, from the input to the output: $\mathbf{v} \mapsto J\mathbf{v}$.
- The **VJP** carries a cotangent back across the function, from the output to the input: $\mathbf{u} \mapsto \mathbf{u}^\top J$.

Pairing a cotangent with a tangent at the same location gives a number. A cotangent is a measuring device, and a tangent is the thing it measures:

- At the **input**: $(8,\ 4) \cdot (1,\ 0.5) = 10$. The input cotangent measures the input tangent.
- At the **output**: $(1,\ 1,\ 1) \cdot (0,\ 2.5,\ 7.5) = 10$. The output cotangent measures the output tangent.

Both give the same 10, because both describe the same thing: the rate at which the score s changes as the inputs move along $\mathbf{v}$ (a step of size ε changes s by about 10ε). That is the identity $\mathbf{u}^\top(J\mathbf{v}) = (\mathbf{u}^\top J)\mathbf{v}$ from Section 11.3, now with actual numbers. The JVP lets you do the pairing at the output end, and the VJP lets you do it at the input end.

A note on wording. People say "the JVP" both for the operation ($J\mathbf{v}$) and for its result, the output tangent. Likewise "the VJP" is used for the operation and for its result, the input cotangent. This guide does the same when the meaning is clear from context.

11.5 Quick glossary

Term	Plain meaning	Symbol in this doc
Perturbation	A small nudge to a value	$\boldsymbol{\delta}$, $\varepsilon\mathbf{v}$
Tangent	A nudge/direction of change (forward mode carries these)	$\mathbf{v}$, $\dot{a}$
Cotangent / covector	A weighting that measures nudges (reverse mode carries these)	$\mathbf{u}$, $\bar{a}$
Pushforward	Moving a tangent through a function (= JVP)	$J\mathbf{v}$
Pullback	Moving a cotangent back through a function (= VJP)	$J^\top\mathbf{u}$
Adjoint	Another word for the cotangent attached to an intermediate value	$\bar{a}$

12. Forward mode vs. reverse mode: which one, and why

12.1 The question this section answers

You have a function and want derivatives from it. Both the JVP and the VJP can give them, so **which one should you use?** The answer depends on the *shape* of the Jacobian. Recall that for $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian J is an $m \times n$ matrix (rows are outputs, columns are inputs).

12.2 The key fact: one pass reads one slice of J

Take the Jacobian of our three-layer network from Sections 8.6 and 9.6 (3 outputs, 2 inputs):

$$J = \begin{bmatrix} 0 & 0 \\ 2 & 1 \\ 6 & 3 \end{bmatrix}$$

- **A forward-mode pass (a JVP)** with $\mathbf{v} = \mathbf{e}_1$ gives the first **column**, $(0, 2, 6)$. With $\mathbf{v} = \mathbf{e}_2$ it gives the second column, $(0, 1, 3)$.
- **A reverse-mode pass (a VJP)** with $\mathbf{u} = \mathbf{e}_1$ gives the first **row**, $(0, 0)$. With $\mathbf{u} = \mathbf{e}_2$ it gives $(2, 1)$, and with $\mathbf{u} = \mathbf{e}_3$ it gives $(6, 3)$.

To read the *whole* matrix:

- Forward mode needs one pass per column, so n **passes** (here $n = 2$).
- Reverse mode needs one pass per row, so m **passes** (here $m = 3$).

Each pass costs about the same (a small multiple of one evaluation of the function), so you want whichever mode needs **fewer passes**.

	Forward mode (JVP)	**Reverse mode (VJP)**
One pass computes	one **column** ($J\mathbf{v}$)	one **row** ($\mathbf{u}^\top J$)
Passes to get the full J	n (one per input)	m (one per output)
Memory	low (no need to store intermediates)	high (must store the forward pass's intermediates)
Best when	few inputs, many outputs ($n \ll m$)	many inputs, few outputs ($n \gg m$)

(Here "$n \ll m$" means "far fewer inputs than outputs", and "$n \gg m$" means the opposite.)

12.3 The shape of J decides

Shape of J	Example	Forward mode	Reverse mode	Winner
Wide (few inputs, many outputs)	3 inputs, 1,000 outputs	3 passes	1,000 passes	forward
Tall (many inputs, few outputs)	10^9 inputs, 1 output	10^9 passes	**1 pass**	**reverse**

Neural network training is the second row. The single output is the loss ($m = 1$) and the inputs are the parameters (billions of them, $n \approx 10^9$). The Jacobian of the loss is one very long row, and a single VJP reads a whole row. Reverse mode gets the entire gradient in **one** backward pass, while forward mode would need **billions** of passes. This is the whole reason backpropagation is reverse mode.

When forward mode wins:

- A function with very few inputs and many outputs, for example a physical simulation with 3 tunable parameters and thousands of outputs. Forward mode needs only 3 passes.
- Computing a single directional derivative $J\mathbf{v}$, which is exactly one JVP.
- Memory-constrained settings where storing the activations (see below) is too costly.

12.4 The memory trade-off

Reverse mode needs fewer passes for a loss, but it has a cost. The backward pass runs *after* the forward pass and uses values from it: the ReLU mask, the input to each linear layer (needed for $\bar{W} = \bar{\mathbf{y}}\,\mathbf{x}^\top$), and so on. So the forward pass's intermediate values (the **activations**) must be stored until the backward pass uses them.

Forward mode has no such requirement: it computes each value and its tangent together, then discards the value as it moves on.

This is why **training uses far more memory than inference**. It is also why techniques like **gradient checkpointing** exist: they store only some of the activations and recompute the rest during the backward pass, trading extra computation for lower memory.

12.5 The same idea as an order of multiplication

This is the content of Sections 8.6 and 9.6 in a compact form. A network's Jacobian is a product of layer Jacobians, $J = J_k \cdots J_2 J_1$, and the two modes differ in *which end they start multiplying from*:

- **Forward mode** multiplies from the **right**: $J_k\Big(\cdots(J_2(J_1\mathbf{v}))\Big)$. It pushes a vector through J_1, then J_2, and so on, in the same direction as the forward pass.
- **Reverse mode** multiplies from the **left**: $\Big(((\mathbf{u}^\top J_k)J_{k-1})\cdots\Big)J_1$. It pushes a vector through J_k first, then J_{k-1}, and so on, in the opposite direction.
- In both cases **each step is a matrix-by-vector product**, which is cheap. Multiplying the matrices together first (matrix-by-matrix products) would be far costlier. This is the whole trick: **never multiply two Jacobians; always multiply a vector through them.**

12.6 Summary

> The shape of J decides the mode. You pay **one pass per column** in forward mode and **one pass per row** in reverse mode. A loss has a single row, so reverse mode wins for training, and its price is storing the forward pass.

---

13. Backpropagation is just a chain of VJPs

Now we connect everything to neural networks.

13.1 Setup

A network with layers computes:

$$\mathbf{x}_0 \xrightarrow{f_1(\cdot;\,\theta_1)} \mathbf{x}_1 \xrightarrow{f_2(\cdot;\,\theta_2)} \mathbf{x}_2 \;\cdots\; \mathbf{x}_k \xrightarrow{\text{loss}} L$$

where $\mathbf{x}_0$ is the input data, θ_i are layer i's parameters, and L is a **scalar**.

We want $\partial L/\partial\theta_i$ for every layer.

Which argument is "the input"? Notice that the network is a function of *two* things: the data $\mathbf{x}_0$ and the parameters $\theta = (\theta_1, \ldots, \theta_k)$. When we talk about "the input" of a Jacobian or a VJP, we mean *whichever argument we choose to differentiate with respect to*, with the other one frozen as a constant. In training, the data and label are fixed and **the variable is** θ: the function we differentiate is $\theta \mapsto L(\theta)$. Section 13.5 walks through this with a digit classifier.

13.2 The backward recursion

Define the cotangent (adjoint) at each activation: $\bar{\mathbf{x}}_i = \partial L/\partial \mathbf{x}_i$.

- Start: $\bar{L} = 1$. Then $\bar{\mathbf{x}}_k = \nabla_{\mathbf{x}_k} L$ (the loss's own gradient).
- For each layer going backward, compute **two VJPs** of that layer's function f_i, using the incoming cotangent $\bar{\mathbf{x}}_i$:

$$\bar{\mathbf{x}}_{i-1} = \left(\frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_{i-1}}\right)^\top \bar{\mathbf{x}}_i \qquad \bar{\theta}_i = \left(\frac{\partial \mathbf{x}_i}{\partial \theta_i}\right)^\top \bar{\mathbf{x}}_i$$

The first VJP passes the signal *back to the previous layer*; the second yields the *parameter gradient*, which is what the optimizer needs.

What are these derivative terms? They are Jacobians, the same kind of matrix as the J we have used throughout. Each layer has *two* of them:

- $\frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_{i-1}} = J_i$ is the Jacobian of layer i with respect to its **input activation**. If $\mathbf{x}_{i-1}$ has p entries and $\mathbf{x}_i$ has q entries, it is a $q \times p$ matrix, with rows indexed by the layer's outputs and columns by its inputs (the convention from Section 5.1). So the first formula, $\bar{\mathbf{x}}_{i-1} = J_i^\top \bar{\mathbf{x}}_i$, is exactly the VJP rule $\bar{\mathbf{a}} = J_g^\top \bar{\mathbf{b}}$ from Section 9.4, with $g = f_i$. The shapes work out: $J_i^\top$ is $p \times q$ and $\bar{\mathbf{x}}_i$ has q entries, so the result has p entries, the size of $\mathbf{x}_{i-1}$. For a linear layer $J_i = W$, and for ReLU J_i is a diagonal 0/1 matrix.
- $\frac{\partial \mathbf{x}_i}{\partial \theta_i}$ is a **different** Jacobian: that of layer i with respect to its **weights**. It has shape $q \times (\text{number of weights in layer } i)$. For a linear layer it is large and highly structured, and its VJP simplifies to the outer product $\bar{W} = \bar{\mathbf{x}}_i \, \mathbf{x}_{i-1}^\top$ from Section 13.3.

So the backward pass applies the transpose of each of the two Jacobians to the same incoming cotangent $\bar{\mathbf{x}}_i$. As always, the framework never builds either matrix: it uses the local rules of Section 13.3.

That is backpropagation. Nothing more. The famous "error signal propagating backward" is the cotangent $\bar{\mathbf{x}}_i$, and "propagating" it means one VJP per layer. Each layer only needs to know how to compute **its own local VJP**; the framework chains them together automatically.

13.3 Local VJP rules for common layers

These are the building blocks that autodiff libraries implement for every primitive operation.

Linear layer $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, with incoming cotangent $\bar{\mathbf{y}}$:

$$\bar{\mathbf{x}} = W^\top \bar{\mathbf{y}} \qquad \bar{W} = \bar{\mathbf{y}}\, \mathbf{x}^\top \qquad \bar{\mathbf{b}} = \bar{\mathbf{y}}$$

Why? $J_{\mathbf{x}} = W$, so the VJP is $W^\top \bar{\mathbf{y}}$. For the weights: entry W_{ij} affects only y_i and does so with multiplier x_j, so $\partial L / \partial W_{ij} = \bar{y}_i \, x_j$. That's the **outer product** $\bar{\mathbf{y}}\mathbf{x}^\top$. (For a batch of examples, the contributions are summed over the batch.)

Elementwise activation $\mathbf{y} = \sigma(\mathbf{z})$: the Jacobian is diagonal, $\mathrm{diag}(\sigma'(\mathbf{z}))$, so the VJP is just an elementwise product:

$$\bar{\mathbf{z}} = \sigma'(\mathbf{z}) \odot \bar{\mathbf{y}}$$

(Here $\odot$ means multiply entry by entry.) For ReLU, $\sigma'(z) = 1$ if $z > 0$ else 0. So ReLU simply *lets the signal pass through where the unit was active, and blocks it elsewhere*. We never build the huge diagonal matrix.

Sum / copy (fan-out): if a value is used in two places, cotangents from both uses **add**. If you add two values, the cotangent flows unchanged to both. This is how skip connections (as in ResNets) work: the gradient takes a "highway" straight back.

Squared-error loss $L = \frac{1}{2}\|\mathbf{h} - \mathbf{t}\|^2$: $\bar{\mathbf{h}} = \mathbf{h} - \mathbf{t}$.

13.4 Why this is efficient

For a network with layers of width d, a naive "build the full Jacobian and multiply" approach costs $O(d^3)$ per layer for matrix-matrix products, and storage is quadratic in size. Backprop does only matrix-**vector** products ($O(d^2)$) and the total backward cost is about **2× the forward pass**. That ratio (parameters' gradients for roughly the price of two evaluations) is what makes large-scale training feasible.

13.5 A concrete example: an MNIST digit classifier

Let's tie everything together with a real task. The input is a 28×28 grayscale image from MNIST, flattened into $\mathbf{x} \in \mathbb{R}^{784}$, and the network outputs 10 scores (one per digit). The question we want to settle: **what exactly is the "input" of the function whose VJP we take?**

Step 1: the network has two arguments

Take a small network: $784 \to 128 \to 10$ with a ReLU in the middle.

$$\mathbf{a} = W_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{h} = \mathrm{ReLU}(\mathbf{a}), \qquad \mathbf{z} = W_2\mathbf{h} + \mathbf{b}_2$$

The 10 numbers $\mathbf{z}$ are the **logits** (raw scores). The function is $\mathbf{z} = N(\theta, \mathbf{x})$, where $\theta = (W_1, \mathbf{b}_1, W_2, \mathbf{b}_2)$ holds all the weights. The number of weights is

$$\underbrace{784 \cdot 128}_{W_1} + \underbrace{128}_{\mathbf{b}_1} + \underbrace{128 \cdot 10}_{W_2} + \underbrace{10}_{\mathbf{b}_2} = 100{,}352 + 128 + 1{,}280 + 10 = 101{,}770$$

N depends on both θ and $\mathbf{x}$. Which one is "the input" for the Jacobian depends on which question we ask. We pick one as the variable, and the other is frozen.

Step 2: training freezes the image and varies the weights

In training we have one image $\mathbf{x}$ with a known label y, and we ask: *"how should I change the weights to make the loss smaller?"* So the function we differentiate is

$$\theta \longmapsto L(\theta) = \text{loss}\Big(N(\theta, \mathbf{x}),\ y\Big)$$

The image and the label are fixed data. **The input of this function is** θ, with $n = 101{,}770$ entries.

Step 3: the loss collapses the 10 outputs into one number

The network has 10 outputs, but the full function ends with a loss layer, for example cross-entropy:

$$L = -\log\Big(\text{softmax}(\mathbf{z})_y\Big)$$

This turns 10 scores into **one** number, so $m = 1$. The Jacobian of L with respect to θ is a single row of $101{,}770$ entries, which is the "tall" case from Section 12. One backward pass (a VJP) gives the whole gradient, while forward mode would need $101{,}770$ passes.

Step 4: where "the weighted sum of the outputs" appears

The backward pass starts at the loss. The loss layer's own VJP produces a cotangent at the logits:

$$\bar{\mathbf{z}} = \frac{\partial L}{\partial \mathbf{z}} = \text{softmax}(\mathbf{z}) - \text{onehot}(y)$$

That is 10 numbers, and it plays exactly the role of the weight vector $\mathbf{u}$ from Section 9 for the rest of the network. For an image of a "3":

- The entry for the correct class (3) is **negative**: raising that logit lowers the loss.
- The entries for the wrong classes are **positive**, and larger for classes the network is more confident about: raising those logits would increase the loss.

From the network's point of view, with θ as the variable and the logits as the 10 outputs, the rest of the backward pass computes the VJP

$$\bar{\mathbf{z}}^\top \frac{\partial \mathbf{z}}{\partial \theta} = \nabla_\theta L$$

This is the gradient of the weighted sum of the outputs $s = \bar{\mathbf{z}}^\top \mathbf{z}$ (with $\bar{\mathbf{z}}$ held fixed), and at this point it equals the gradient of L. You never choose $\mathbf{u}$ by hand: the loss layer's backward step computes it.

Step 5: the layers, one VJP at a time

Starting from $\bar{\mathbf{z}}$ (10 numbers), apply the local rules of Section 13.3, last layer first:

Layer	Cotangent in	Computation	Output	Shape
Linear 2 ($\mathbf{z} = W_2\mathbf{h} + \mathbf{b}_2$)	$\bar{\mathbf{z}}$	$\bar{W}_2 = \bar{\mathbf{z}}\,\mathbf{h}^\top,\ \bar{\mathbf{b}}_2 = \bar{\mathbf{z}}$	weight gradients	10×128, 10
		$\bar{\mathbf{h}} = W_2^\top \bar{\mathbf{z}}$	goes back	128
ReLU ($\mathbf{h} = \mathrm{ReLU}(\mathbf{a})$)	$\bar{\mathbf{h}}$	$\bar{\mathbf{a}} = [\mathbf{a} > 0] \odot \bar{\mathbf{h}}$	goes back	128
Linear 1 ($\mathbf{a} = W_1\mathbf{x} + \mathbf{b}_1$)	$\bar{\mathbf{a}}$	$\bar{W}_1 = \bar{\mathbf{a}}\,\mathbf{x}^\top,\ \bar{\mathbf{b}}_1 = \bar{\mathbf{a}}$	weight gradients	128×784, 128
		$\bar{\mathbf{x}} = W_1^\top \bar{\mathbf{a}}$	image sensitivity	784

Every weight gradient comes out of **one** backward pass: $1{,}280 + 10 + 100{,}352 + 128 = 101{,}770$ numbers, exactly the size of θ. Each layer's two VJPs are the two formulas of Section 13.2: one passes the cotangent back to the previous layer, and the other produces that layer's weight gradient. The picture:

```
forward:   x (784 pixels) --> Linear1 --> ReLU --> Linear2 --> z (10 logits)
--> loss L

|
backward:  grad W1,b1 <-- ReLU mask <-- grad W2,b2 <-- z_bar = softmax(z) -
onehot(y)
```

Then gradient descent updates every weight: $\theta \leftarrow \theta - \eta\,\bar{\theta}$.

Notice the last row of the table: the pass also computes $\bar{\mathbf{x}}$, the sensitivity of the loss to each of the 784 pixels. During training this is simply thrown away.

Step 6: the other choice, freezing the weights

If we freeze the trained weights and make the **image** the variable, the function is $\mathbf{x} \mapsto L(\mathbf{x})$, and the same backward pass gives $\bar{\mathbf{x}} = \nabla_{\mathbf{x}} L$, the exact quantity thrown away in training. It is used for:

- **Adversarial examples:** nudge the pixels in the direction that increases the loss, so the network misclassifies.
- **Saliency maps:** which pixels is the prediction most sensitive to?

It is the same machinery with a different choice of variable.

Summary

View	Variable (the "input" of f)	Frozen	n	m	What the VJP gives
Training	weights θ	image, label	$101{,}770$	1 (the loss)	$\nabla_\theta L$, used to update the weights
Adversarial / saliency	pixels $\mathbf{x}$	weights, label	784	1 (the loss)	$\nabla_\mathbf{x} L$, the sensitivity of the loss to each pixel

Both views have a single output, the loss, so both need just one backward pass. The theory never changes: **you choose which argument to differentiate with respect to, and the VJP gives the sensitivity of the score to that argument.**

In practice we train on batches of images and average the loss. The weights are shared across the batch, so each weight's gradient is the sum (or average) of the per-image gradients, as noted in Section 13.3.

---

14. A complete worked backprop example, by hand

A tiny one-layer network, so you can follow every number.

Setup:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & -1 \\ 2 & 0.5 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{t} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \qquad \mathbf{h} = \mathrm{ReLU}(\mathbf{z}), \qquad L = \tfrac{1}{2}\|\mathbf{h} - \mathbf{t}\|^2$$

Forward pass

- $W\mathbf{x} = (1 \cdot 1 + (-1) \cdot 2,\ 2 \cdot 1 + 0.5 \cdot 2) = (-1,\ 3)$
- $\mathbf{z} = (-1 + 0,\ 3 - 1) = (-1,\ 2)$
- $\mathbf{h} = \mathrm{ReLU}(\mathbf{z}) = (0,\ 2)$
- $L = \tfrac{1}{2}\left[(0-1)^2 + (2-1)^2\right] = \tfrac{1}{2}(1+1) = 1$

Backward pass (a chain of VJPs, last layer first)

1. **Loss:** $\bar{\mathbf{h}} = \mathbf{h} - \mathbf{t} = (-1,\ 1)$.
2. **ReLU:** the mask is $[z > 0] = (0, 1)$. So $\bar{\mathbf{z}} = (0, 1) \odot (-1, 1) = (0,\ 1)$. (Neuron 1 was "dead" so it gets no gradient.)
3. **Linear layer:**

- $\bar{\mathbf{b}} = \bar{\mathbf{z}} = (0,\ 1)$
- $\bar{W} = \bar{\mathbf{z}}\,\mathbf{x}^\top = \begin{bmatrix}0\\1\end{bmatrix}\begin{bmatrix}1 & 2\end{bmatrix} = \begin{bmatrix}0 & 0\\1 & 2\end{bmatrix}$
- $\bar{\mathbf{x}} = W^\top \bar{\mathbf{z}} = \begin{bmatrix}1 & 2\\-1 & 0.5\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}2\\0.5\end{bmatrix}$

Sanity check with a finite difference

Nudge W_{21} (row 2, column 1) from 2 to 2.001. Then $z_2 = 2.001 \cdot 1 + 0.5 \cdot 2 - 1 = 2.001$, $h_2 = 2.001$, and $L = \frac{1}{2}(1 + 1.001^2) = \frac{1}{2}(1 + 1.002001) = 1.0010005$. The change is 0.0010005, so the slope $\approx 1.0005 \approx 1$, matching $\bar{W}_{21} = 1$.

Gradient descent step

With learning rate $\eta = 0.1$: $W \leftarrow W - 0.1\,\bar{W} = \begin{bmatrix}1 & -1\\1.9 & 0.3\end{bmatrix}$, $\mathbf{b} \leftarrow (0,\ -1.1)$. The new $z_2 = 1.9 + 0.6 - 1.1 = 1.4$, so $h_2 = 1.4$, which is closer to the target 1. The loss dropped. That's learning.

---

15. The Hessian: the derivative of the derivative

15.1 Motivation

The gradient tells you the slope at a point. But slope alone doesn't tell you **how far** you can safely step, or **whether you're near a bottom**. For that you need to know how the slope *itself* is changing: **curvature**. In one dimension that's the second derivative $f''(x)$:

- $f'' > 0$: curves upward (a bowl).
- $f'' < 0$: curves downward (a hill).
- $f'' = 0$: no curvature at that point (locally flat), and the second derivative alone cannot tell you what the function does next.

At a flat spot of the graph (where $f' = 0$), these signs decide whether you are at the bottom of a bowl (a minimum), the top of a hill (a maximum), or neither. The Hessian is the multi-dimensional version of f''.

15.2 Definition

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the **Hessian** is the $n \times n$ matrix of all second partial derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i\,\partial x_j}$$

Equivalently and importantly:

$$\boxed{H = \text{Jacobian of the gradient}}$$

The gradient $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$ is itself a vector-valued function. Its Jacobian is $n \times n$, and that is the Hessian. This identity is how we'll compute Hessian-related quantities with the JVP/VJP machinery from earlier. In particular, the gradient is itself computed by a VJP (Section 9.8), so the Hessian is "the Jacobian of a VJP." Section 16 uses exactly this fact to get Hessian-vector products without ever building H.

Symmetry: if the second derivatives are continuous (true for the smooth functions in this guide), then $\partial^2 f / \partial x_i \partial x_j = \partial^2 f / \partial x_j \partial x_i$, so $H = H^\top$.

15.3 Example

Take $f(x, y) = x^2 y + y^2$. Gradient: $(2xy,\ x^2 + 2y)$. Differentiate each component again:

$$H = \begin{bmatrix} \partial_x(2xy) & \partial_y(2xy) \\ \partial_x(x^2 + 2y) & \partial_y(x^2 + 2y) \end{bmatrix} = \begin{bmatrix} 2y & 2x \\ 2x & 2 \end{bmatrix}$$

At $(1, 2)$: $H = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$. Note the off-diagonals match (symmetric).

The trace is 6 and the determinant is $4 \cdot 2 - 2 \cdot 2 = 4$, so the eigenvalues are $3 \pm \sqrt{5} \approx 5.24$ and 0.76. Both are positive, so **at** $(1, 2)$ **the function curves upward in every direction**, much more steeply along one direction than the other (a narrow valley). Note that this is a statement about curvature only: $\nabla f(1, 2) = (4, 5) \neq \mathbf{0}$, so $(1, 2)$ is *not* a minimum. We are on the side of the valley, not at the bottom.

15.4 What the Hessian tells you: the second-order approximation

Extending Section 2's linear approximation by one more term:

$$f(\mathbf{x} + \boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f^\top \boldsymbol{\delta} + \tfrac{1}{2}\, \boldsymbol{\delta}^\top H\, \boldsymbol{\delta}$$

The last term is the **curvature correction**. Its sign and size depend on the direction $\boldsymbol{\delta}$:

- $\boldsymbol{\delta}^\top H \boldsymbol{\delta}$ is the curvature of f along direction $\boldsymbol{\delta}$ (for unit-length $\boldsymbol{\delta}$).
- The **eigenvalues** of H are the curvatures along its principal axes (its **eigenvectors**).

Numeric check (the same kind of check we did for the gradient in Section 4.3). At $(1, 2)$ with $\boldsymbol{\delta} = (0.1, -0.1)$:

- Exact: $f(1.1,\ 1.9) = 1.21 \cdot 1.9 + 1.9^2 = 2.299 + 3.61 = 5.909$, and $f(1, 2) = 6$.
- Linear approximation: $6 + \nabla f^\top \boldsymbol{\delta} = 6 + (4 \cdot 0.1 + 5 \cdot (-0.1)) = 5.9$. The error is 0.009.

- Curvature term: $\boldsymbol{\delta}^\top H \boldsymbol{\delta} = 4(0.01) + 2 \cdot 2 \cdot (0.1)(-0.1) + 2(0.01) = 0.02$, so $\frac{1}{2}\boldsymbol{\delta}^\top H \boldsymbol{\delta} = 0.01$.
- Second-order approximation: $5.9 + 0.01 = 5.91$. The error drops to 0.001.

Adding the Hessian term made the local model nine times more accurate here.

15.5 Classifying a critical point

A **critical point** is a point where $\nabla f = \mathbf{0}$ (the slope is zero in every direction). There, the linear term vanishes and the Hessian decides the shape:

Eigenvalues of H	**Shape**	**Meaning**
all positive	bowl	local **minimum**
all negative	dome	local **maximum**
mixed signs (some positive, some negative)	saddle	neither: goes up in some directions and down in others
some zero, the rest all ≥ 0 (or all ≤ 0)	flat directions	**inconclusive**: the Hessian alone cannot decide

Examples at the origin (where the gradient is zero in each case):

- $f = x^2 + 3y^2$ has $H = \mathrm{diag}(2, 6)$: a minimum.
- $f = x^2 - y^2$ has $H = \mathrm{diag}(2, -2)$: a saddle, like a mountain pass.
- $f = x^2 + y^4$ and $f = x^2 + y^3$ both have $H = \mathrm{diag}(2, 0)$ at the origin, yet the first has a minimum there and the second does not (along the y axis, y^3 is negative for $y < 0$). The Hessian alone cannot distinguish them, which is exactly what "inconclusive" means.

Why this matters in AI: the Hessians of trained neural networks typically have a huge number of eigenvalues near zero (flat directions) and a small number of large ones. So the inconclusive case is the normal case in deep learning, not an edge case.

15.6 Gradient descent on a quadratic: where the learning-rate limit comes from

A quadratic bowl is the simplest landscape with curvature. Take

$$f(x, y) = \tfrac{1}{2}\left(x^2 + 10y^2\right), \qquad \nabla f = (x,\ 10y), \qquad H = \begin{bmatrix} 1 & 0 \\ 0 & 10 \end{bmatrix}$$

The eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = 10$: very different curvatures, a narrow valley. Gradient descent updates each coordinate independently:

$$x \leftarrow x - \eta \cdot x = (1 - \eta \cdot 1)\, x, \qquad y \leftarrow y - \eta \cdot 10y = (1 - \eta \cdot 10)\, y$$

In general, along an eigen-direction with eigenvalue λ, **each step multiplies the error by** $(1 - \eta\lambda)$. The error shrinks only if $|1 - \eta\lambda| < 1$, i.e. $0 < \eta < 2/\lambda$. This must hold for *every* direction, so

$$\boxed{\eta < \frac{2}{\lambda_{\max}}}$$

This is the learning-rate rule quoted in Section 17. Starting from $(1, 1)$, 50 steps give:

Learning rate η	**x factor per step**	**y factor per step**	**After 50 steps**
0.10	0.9	0	$x \approx 0.005,\ y = 0$
0.19	0.81	-0.9	$x \approx 3 \times 10^{-5},\ y \approx 0.005$
0.21 (too large)	0.79	-1.1	$x \approx 0$, $y \approx 117$ (blown up)

Two lessons from this table:

1. **The steepest direction limits how big a step is safe.** At $\eta = 0.21$ the y direction ($\lambda = 10$) overshoots more each time (factor -1.1) and diverges.
2. **The flattest direction limits how fast you converge.** At $\eta = 0.1$, y is solved immediately but x (with $\lambda = 1$) shrinks by only 0.9 per step. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ (here 10), called the **condition number**, measures how bad this mismatch is. The larger κ is, the slower gradient descent gets, because the step is held back by the steepest direction while progress is needed in the flattest.

15.7 Newton's method

Minimize the second-order approximation over $\boldsymbol{\delta}$: set its gradient to zero, giving $H\boldsymbol{\delta} = -\nabla f$, i.e.

$$\boldsymbol{\delta} = -H^{-1}\nabla f$$

That's **Newton's method**: it uses curvature to choose both the direction *and* the step length. It divides the gradient by the curvature in each direction, so steep and flat directions are treated alike.

Worked example. On the valley $f = \frac{1}{2}(x^2 + 10y^2)$ from $(1, 1)$: $\nabla f = (1, 10)$ and H^{-1} $= \mathrm{diag}(1, \frac{1}{10})$, so

$$\boldsymbol{\delta} = -\left(1 \cdot 1,\ \tfrac{1}{10} \cdot 10\right) = (-1,\ -1) \quad \Longrightarrow \quad (1, 1) + (-1, -1) = (0, 0)$$

One step lands exactly on the minimum. In any perfect quadratic bowl, Newton's method does this, whereas plain gradient descent needed dozens of steps in the table above and could

not safely use a large learning rate.

A caveat. Setting the gradient of the approximation to zero finds a *critical point of the approximation*, which is a minimum only if H is positive definite (all eigenvalues positive). Where H has negative eigenvalues, Newton's method heads toward a saddle or a maximum instead. For non-convex problems like neural networks, practical versions modify or damp H (for example, adding a multiple of the identity) to keep the step sensible.

15.8 The scale problem

For n parameters, H has n^2 entries and inverting it costs $O(n^3)$. Take the small MNIST network from Section 13.5, with $n = 101{,}770$ weights:

- H has $101{,}770^2 \approx 1.04 \times 10^{10}$ entries, about **41 GB** in 32-bit floats. That is for a *tiny* network.
- A model with $n = 10^9$ parameters would have a Hessian with 10^{18} entries.

Building and inverting H is impossible at these sizes. Hence the next section: we never need H itself, only what it does to a vector.

> **Takeaway:** the Hessian is the Jacobian of the gradient. Its eigenvalues are curvatures: they classify critical points, set the largest safe learning rate ($2/\lambda_{\max}$), and determine how fast gradient descent converges ($\kappa = \lambda_{\max}/\lambda_{\min}$). Newton's method uses H to fix all of that in one step on a quadratic, but at a cost that rules it out for large models.

---

16. Hessian-vector products: using curvature without building it

16.1 The key trick

Just like with Jacobians, we rarely need the Hessian itself, only what it does to a vector $\mathbf{v}$:

$$H\mathbf{v}$$

Since H is the Jacobian of the gradient function $g(\mathbf{x}) = \nabla f(\mathbf{x})$, we have

$$H\mathbf{v} = J_g\,\mathbf{v} = \text{JVP of } g \text{ in direction } \mathbf{v} = \frac{d}{d\varepsilon}\nabla f(\mathbf{x} + \varepsilon\mathbf{v})\Big|_{\varepsilon=0}$$

In words: **nudge the point along $\mathbf{v}$ and see how the gradient changes.** That rate of change of the gradient is $H\mathbf{v}$.

16.2 How it is computed

The gradient g itself is computed by a VJP (backprop). So an HVP is **a JVP applied on top of a VJP**: "forward-over-reverse" differentiation. The cost is roughly a small constant multiple of one gradient computation (about 2 to 3 times, as a rule of thumb), in memory and time, **without ever forming the $n \times n$ matrix**.

In practice you never write ε explicitly. The JVP machinery of Section 8 computes that derivative exactly, as a by-product of evaluating the gradient.

An alternative: reverse-over-reverse. Because H is symmetric, $H\mathbf{v} = (\mathbf{v}^\top H)^\top$. And $\mathbf{v}^\top H$ is the gradient of the *scalar* function $\mathbf{x} \mapsto \mathbf{v}^\top \nabla f(\mathbf{x})$, which an ordinary backward pass can compute. So one can also get $H\mathbf{v}$ by running backprop twice. Forward-over-reverse is typically cheaper, so it is the usual choice. A code example of both the Hessian and the HVP is in Section 18.1.

16.3 Example

Take $f = x^2 y + y^2$ at $(1, 2)$ with $\mathbf{v} = (1, 0)$. The gradient at the nudged point is

$$\nabla f(\mathbf{x} + \varepsilon \mathbf{v}) = \Big(2(1+\varepsilon)(2),\ (1+\varepsilon)^2 + 4\Big) = \Big(4 + 4\varepsilon,\ 5 + 2\varepsilon + \varepsilon^2\Big)$$

Differentiating in ε at 0 gives $(4,\ 2)$. And from the full Hessian,

$$H\mathbf{v} = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = (4, 2)$$

The two agree. The first route never needed the matrix H.

16.4 What HVPs enable

- **Conjugate gradient** / Newton-CG: solve $H\boldsymbol{\delta} = -\nabla f$ iteratively using only HVPs.
- **Power iteration / Lanczos:** estimate the dominant eigenvalue of H (the one largest in magnitude, which for loss landscapes is usually the largest positive one, the **sharpness**) by repeatedly computing $H\mathbf{v}$.
- **Hutchinson's trick:** estimate $\mathrm{trace}(H)$ via $\mathbb{E}[\mathbf{z}^\top H \mathbf{z}]$, where $\mathbf{z}$ is a random vector whose entries are independent with mean 0 and variance 1 (typically random ± 1 entries), using one HVP per sample.

> **Takeaway:** an HVP is a JVP of the gradient. It gives curvature along a direction at about the cost of a few gradient computations, so second-order information becomes available even when H is far too large to build.

---

17. Where all this shows up in AI

The ideas of this guide are not just background theory: each one has a specific job in modern machine learning. The table gives a map, with pointers to the sections that explain each idea. Sections 17.1 and 17.2 add detail on the two items that need it most.

Idea	Where it's used	Read more
Gradient (via VJP / backprop)	Training every neural network: SGD, Adam, etc. Each step is "compute ∇L with one backward pass, then update weights."	Sections 9, 13
VJP with respect to the inputs	The gradient of any scalar quantity with respect to any argument: adversarial examples (gradient of the loss with respect to the *input image*), saliency maps, style transfer, and soft-prompt tuning.	Section 13.5
Jacobians of layers	Understanding signal flow: the Jacobians of the layers control whether gradients **explode** or **vanish** through depth. This motivates ReLU, residual connections, careful initialization and normalization layers.	Sections 6, 17.2
JVP	Directional derivatives, curvature products, kernel analysis, and some training schemes that avoid backprop.	Sections 8, 17.1
Hessian / HVP	Second-order optimizers; learning-rate selection (a stable step size for gradient descent on a quadratic is $< 2/\lambda_{\max}$); *measuring* sharpness (the largest Hessian eigenvalue, found by power iteration); saddle-point analysis; model pruning (Optimal Brain Surgeon uses Hessian information); influence functions (which training examples affected a prediction); Laplace approximations for uncertainty.	Sections 15, 16
Jacobian determinants	Normalizing flows use $\det J$ to track how probability density changes under a transformation.	Section 5
Meta-learning	Second-order MAML needs gradients *of gradients* (HVPs). (First-order MAML skips them.)	Section 16
Implicit differentiation	Differentiating through optimization problems or fixed points: the VJP is obtained by solving a linear system with the transposed Jacobian of the defining equation.	Section 9

A note on sharpness: *sharpness-aware minimization* (SAM) aims for flat minima, but it uses **only gradients**. It takes a small step uphill, then computes the gradient at that perturbed point. It involves no Hessian and no HVP. HVPs enter when you want to *measure* sharpness, as in the table.

17.1 The JVP in more detail

The JVP shows up in several places, and sometimes together with a VJP:

- **Directional derivatives and forward-mode sensitivity.** "How does everything change if I nudge the inputs this way?" is exactly one JVP (Section 8).
- **Gauss-Newton products.** The Gauss-Newton matrix approximates the Hessian of a loss through a network as $G = J_\theta^\top H_{\text{out}} J_\theta$, where J_θ is the Jacobian of the network's outputs with respect to its weights and H_{out} is the Hessian of the loss with respect to the outputs. A product with a vector is

  $$G\mathbf{v} = J_\theta^\top \Big(H_{\text{out}} \Big(J_\theta \mathbf{v} \Big) \Big),$$

  which takes **a JVP first and then a VJP**, so G is never built. Hessian-free optimization uses these products. (K-FAC approximates the same kind of matrix with small Kronecker-product blocks built from per-layer statistics, and does not compute these products.)
- **Neural tangent kernel (NTK) analysis.** The NTK is built from the Jacobian of the output with respect to the parameters ($J_\theta J_\theta^\top$). JVPs and VJPs compute products with it without forming it.
- **Forward gradients.** Pick a random direction $\mathbf{v}$, compute the JVP $\nabla f^\top \mathbf{v}$ (one forward-mode pass, no stored activations), and use $(\nabla f^\top \mathbf{v})\,\mathbf{v}$ as an estimate of the gradient. It is unbiased but **noisy**, so it trades memory for a harder optimization problem.
- **Differentiating through ODE solvers.** Forward sensitivity analysis is JVP-style. The *adjoint method* used by Neural ODEs is VJP-based and is usually preferred when there are many parameters.

17.2 Why "vanishing gradients" is a Jacobian statement

The backward signal at layer i is

$$\bar{\mathbf{x}}_i = J_{i+1}^\top J_{i+2}^\top \cdots J_k^\top \, \bar{\mathbf{x}}_k$$

a product of many matrices, where $J_j = \partial \mathbf{x}_j / \partial \mathbf{x}_{j-1}$ is layer j's Jacobian (Section 13.2). The size of this product, not of any single factor, decides what happens. The norm of a product is at most the product of the norms, which gives the simplest picture:

- If **every** Jacobian scales every vector by at most 0.5, then after 50 layers the signal is at most $0.5^{50} \approx 9 \times 10^{-16}$ of the original. It **vanishes**.
- If **every** Jacobian scales every vector by at least 2, then it grows by at least $2^{50} \approx 1.1 \times 10^{15}$. It **explodes**.

Both are disastrous. Real Jacobians do not scale all vectors equally (their singular values differ), so the actual behavior depends on the singular values of the *whole product*. But the rough picture is right: **repeated multiplication by matrices of size far from 1 is unstable.** A

great deal of deep learning architecture design is about keeping those Jacobian products well-behaved.

> **Takeaway:** VJPs give the gradients that train models, JVPs and HVPs give directional and curvature information, and the Jacobians of the layers decide whether those gradients arrive intact.

---

18. Seeing it in code (JAX and PyTorch)

JAX exposes JVP and VJP as first-class functions with exactly the names used here.

18.1 JAX

```
import jax
import jax.numpy as jnp

def f(x):
    return jnp.array([x[0] * x[1],
                      x[0] + x[1] ** 2,
                      jnp.sin(x[0])])

x = jnp.array([1.0, 2.0])

# --- JVP: push a tangent forward -----------------------------------
v = jnp.array([1.0, 0.5])                       # input-sized tangent
y, jv = jax.jvp(f, (x,), (v,))                  # jv = J @ v
print("JVP:", jv)                               # [2.5, 3.0, 0.5403]

# --- VJP: pull a cotangent backward --------------------------------
y, vjp_fn = jax.vjp(f, x)
u = jnp.array([1.0, 1.0, 1.0])                  # output-sized cotangent
(vj,) = vjp_fn(u)                               # vj = J.T @ u
print("VJP:", vj)                               # [3.5403, 5.0]

# --- Full Jacobian (built from many JVPs or VJPs) ------------------
J_fwd = jax.jacfwd(f)(x)   # n JVPs (good when few inputs)
J_rev = jax.jacrev(f)(x)   # m VJPs (good when few outputs)
print(J_fwd)               # 3x2 matrix; J_rev is identical

# --- Gradient of a scalar = VJP with u = 1 -------------------------
def loss(x):
    return x[0] ** 2 * x[1] + x[1] ** 2
print(jax.grad(loss)(x))                        # [4., 5.]
```

```
# --- Hessian, and Hessian-vector product --------------------------
H = jax.hessian(loss)(x)                          # [[4, 2], [2, 2]]

def hvp(fun, x, v):
    # forward-over-reverse: JVP of the gradient function
    return jax.jvp(jax.grad(fun), (x,), (v,))[1]

print(hvp(loss, x, jnp.array([1.0, 0.0])))      # [4., 2.]  (== H @ v)
```

18.2 PyTorch

```
import torch
from torch.autograd.functional import jvp, vjp, jacobian, hessian

def f(x):
    return torch.stack([x[0] * x[1],
                        x[0] + x[1] ** 2,
                        torch.sin(x[0])])

x = torch.tensor([1.0, 2.0])

# JVP and VJP
_, jv = jvp(f, x, torch.tensor([1.0, 0.5]))
_, vj = vjp(f, x, torch.tensor([1.0, 1.0, 1.0]))
print(jv, vj)

# Full Jacobian and Hessian (small problems only!)
print(jacobian(f, x))

def loss(x):
    return x[0] ** 2 * x[1] + x[1] ** 2
print(hessian(loss, x))

# The everyday case: backprop IS a VJP with u = 1 on a scalar loss
w = torch.tensor([1.0, 2.0], requires_grad=True)
L = loss(w)
L.backward()           # runs the backward pass (chain of VJPs)
print(w.grad)          # tensor([4., 5.])
```

When you call `L.backward()`, PyTorch is running exactly the reverse-mode procedure of Sections 9 and 13, starting from the cotangent $\bar{L} = 1$.

18.3 A habit worth building: check against finite differences

```
def numeric_jvp(f, x, v, eps=1e-3):
    return (f(x + eps * v) - f(x - eps * v)) / (2 * eps)   # central
difference
```

Compare `numeric_jvp(f, x, v)` with the JVP from the framework (and, for a gradient, check `g @ v` against it). If they disagree by more than the tolerances below, you have a bug.

Precision matters. JAX and PyTorch use 32-bit floats by default, which carry about 7 digits. The numerator of a finite difference subtracts two nearly equal numbers, so the rounding error is roughly $10^{-7}/\texttt{eps}$. Making `eps` too small makes the check *worse*. In our example (the function from 18.1):

Precision	`eps`	Max relative error (correct code)
float32	10^{-4}	about 6×10^{-4}
float32	10^{-3}	about 3×10^{-5}
float64	10^{-4}	about 2×10^{-9}
float64	10^{-5}	about 2×10^{-11}

Rules of thumb:

- In **float32**, use `eps` around 10^{-3} and treat a relative error above about 10^{-3} as suspicious.
- For a tighter check, switch to **float64** (in JAX: `jax.config.update("jax_enable_x64", True)`; in PyTorch: create tensors with `dtype=torch.float64`), use `eps` around 10^{-5}, and treat a relative error above about 10^{-6} as suspicious.

---

19. Cheat sheet

Shapes (for $f : \mathbb{R}^n \to \mathbb{R}^m$):

Object	Shape	Meaning
J	$m \times n$	$J_{ij} = \partial f_i / \partial x_j$
JVP: $J\mathbf{v}$	$\mathbf{v} \in \mathbb{R}^n \Rightarrow \mathbb{R}^m$	output change per input nudge $\mathbf{v}$
VJP: $\mathbf{u}^\top J$ (or $J^\top \mathbf{u}$)	$\mathbf{u} \in \mathbb{R}^m \Rightarrow \mathbb{R}^n$	input sensitivity of $\mathbf{u}^\top f$
Gradient of scalar f	n	VJP with $u = 1$
Hessian of scalar f	$n \times n$	J of ∇f, symmetric
HVP: $H\mathbf{v}$	n	JVP of ∇f

The four objects (Section 11.4):

Object	Name	Lives at
$\mathbf{v}$	input tangent	input
$J\mathbf{v}$	output tangent	output
$\mathbf{u}$	output cotangent	output
$\mathbf{u}^\top J$	input cotangent	input

Rules of thumb:

- Chain rule: $J_{g\circ f} = J_g J_f$ (later function on the left).
- Forward mode: carries **tangents** forward (multiplying by J, right to left), reads one column of J per pass, cheap when inputs are few.
- Reverse mode: carries **cotangents** backward (multiplying by $J^\top$, left to right), reads one row of J per pass, cheap when outputs are few. Needs stored activations.
- Never multiply two Jacobians together and never build J or H for large models; always push a vector through.
- Backprop = reverse mode on a scalar loss = a chain of local VJPs.
- Linear layer: $\bar{\mathbf{x}} = W^\top \bar{\mathbf{y}},\ \bar{W} = \bar{\mathbf{y}}\,\mathbf{x}^\top,\ \bar{\mathbf{b}} = \bar{\mathbf{y}}$.
- Elementwise activation: $\bar{\mathbf{z}} = \sigma'(\mathbf{z}) \odot \bar{\mathbf{y}}$.
- If a value is used multiple times, its cotangents **add**.
- "The input" is whichever argument you choose to differentiate with respect to. In training it is the weights θ, with the data frozen (Section 13.5).
- $H\mathbf{v}$ = JVP of the gradient function (forward-over-reverse).
- Duality identity: $\mathbf{u}^\top(J\mathbf{v}) = (\mathbf{u}^\top J)\mathbf{v}$.

Curvature (Sections 15 and 16):

- Second-order approximation: $f(\mathbf{x}+\boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top H \boldsymbol{\delta}$.
- At a critical point ($\nabla f = \mathbf{0}$): all eigenvalues of H positive means a minimum, all negative a maximum, mixed signs a saddle, and zero eigenvalues are inconclusive.
- Gradient descent on a quadratic is stable only if $\eta < 2/\lambda_{\max}$. Its speed is limited by the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$.
- Newton's step is $\boldsymbol{\delta} = -H^{-1}\nabla f$. It reaches the minimum of a quadratic in one step, but it is a descent step only if H is positive definite.
- H has n^2 entries, so for large models use HVPs, never the matrix itself.

Checking your work (Section 18.3): compare against a central finite difference. In float32 use $\texttt{eps} \approx 10^{-3}$ and expect agreement to about 10^{-3}; in float64 use $\texttt{eps} \approx 10^{-5}$ and expect about

10^{-6}.

If you remember only one thing: **a derivative is a machine that turns input nudges into output nudges. A JVP runs that machine forward on a nudge; a VJP runs its transpose backward on a sensitivity; backpropagation is a chain of VJPs; and the Hessian is what you get when you differentiate the derivative itself.**